Lesson Notes

MODULE 6 | LESSON 4

ADVANCED FACTOR CONSTRUCTION

Reading Time	30 minutes
Prior Knowledge	Traditional portfolio theory and utility theory.
Keywords	behavioral finance, bounded rationality, satisfice, prospect theory, cognitive biases, emotional biases.

In the last lesson, we saw that neural networks (especially shallow ones) performed very well compared to other machine learning models, deftly sorting out the most predictive variables and handling their interactions. In this lesson, we see neural nets put to work again. This time, however, we see neural nets go beyond simply finding the best variables and combining them in the best model; the neural nets in this lesson actually construct new factors using the preexisting factors such as technical indicators as building blocks.

1. What is Factor Construction?

In the last lesson, we used various machine learning techniques to build factor models. As part of that process, we selected the most predictive variables from a wide array of variables that we already had reason to believe might add value. Now, instead of factor *selection*, we are concerned with factor *construction*. In the research we explore as our required reading for this lesson, the authors Fang et al. state that the more traditional factor selection process is relatively manual in that each factor needs to be evaluated and regularly updated, which is itself very time consuming. What's worse is that the process of factor selection can be very complex when it is necessary to incorporate potential factor interactions among a large number of potential factors, as we saw in the last lesson. For these reasons, state-of-the-art research and practice in factor models has started to deploy genetic programming (GP) to construct factors automatically, that is, a process known as "Automatic Feature Construction" (2).

2. Genetic Programming

By now, you have been exposed to some evolutionary programming algorithms. To supplement what you have already learned, the following description of genetic programming should suffice for your full comprehension of this lesson's main required reading:

> "Genetic Programming (GP) is a type of Evolutionary Algorithm (EA), a subset of machine learning. EAs are used to discover solutions to problems humans do not know how to solve, directly. Free of human preconceptions or biases, the adaptive nature of EAs can generate solutions that are comparable to, and often better than the best human efforts.
> Inspired by biological evolution and its fundamental mechanisms, GP software systems implement an algorithm that uses random mutation, crossover, a fitness function, and multiple generations of evolution to resolve a user-defined task. GP can be used to discover a functional relationship between features in data (symbolic regression), to group data into categories (classification), and to assist in the design of electrical circuits, antennae, and quantum algorithms." (*Genetic Programming)*

GP is a fascinating and fruitful technique for these reasons. In order to fully appreciate how GP is used by the authors, both as a benchmark and as a foundation upon which to innovate, it is important that you are familiar with the algorithm. To that end, the three basic steps (and their sub-steps) proposed by Langdon et al. are as follows:

1. Generate an initial, stochastic population.
2. Iteratively perform selection, genetic operation, and evaluation:
    - Evaluate each program (hypothetical models) in the current population, with this value recorded as a fitness score.
    - Randomly select programs and compare their fitness scores.
    - Apply one of the genetic operators to a copy of the leading program:
        - mutation
        - crossover

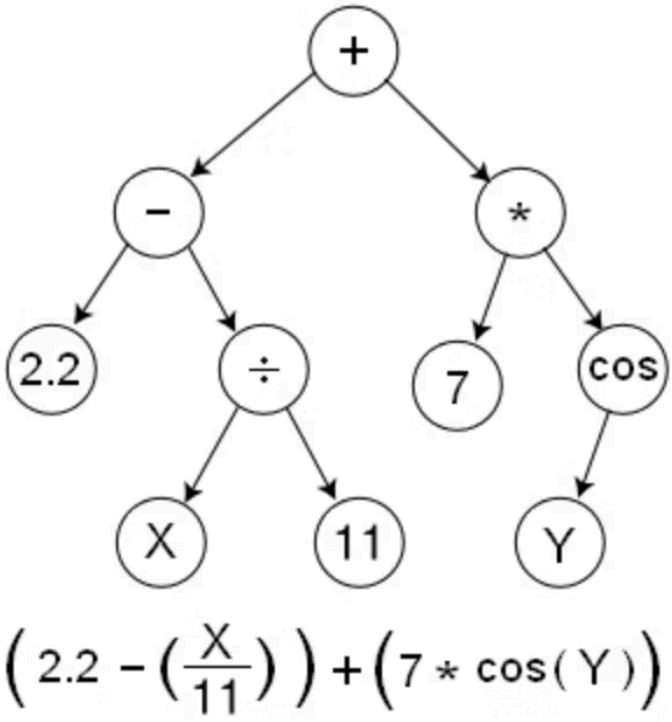
… then move the program into the subsequent generation.

1. Evaluate all programs (hypothetical models) in each new generation.
2. Repeat until the user-defined termination criteria are met.

3. Tree-Based Genetic Programming

In the tree-based GP used as a baseline in the paper, the "population" referred to in step 1 of the above GP algorithm is a population of trees, where "each tree is an explicit formula" (Fang et al. 5). The "genetic operation" refers to changing parts of this formula, either the variables (like price or volume, etc.) or the operators (addition, subtraction, division, multiplication, etc.) or parts of the tree that include both. For example, in the diagram below, the part of the tree connected to the left node that consists of division operator, the variable X, and the scalar 11 could all be replaced by another combination of operator and two operands (a combination of variables or numbers).

Figure 1: Example of Syntax-Tree Structure in Genetic Programming



Source: Genetic Programming, https://geneticprogramming.com/about-gp/tree-based-gp/

Do not be confused by the terminology reverse polish notation (RPN). It just means that the operator (e.g., division) follows the operands (e.g., X, 11) when the equation is written out: " X 11 /." The main point is that we can swap out one part of the model formula for another, where the one that we substitute in can be the result of crossover or mutation as listed in step 2.3. In crossover, parts of the well-performing trees are swapped with each other, to see if the new combination of tree parts perform even better. In mutation, parts of the well-performing tree are changed randomly; the point is to introduce diversity into the new population.

Note, however, that Fang finds that GP results in “a large number of very similar factors, which do not contain new information” (2). In other words, there is insufficient diversity to improve the models. Such “highly correlated outputs” (Fang 2) can be expected to create circumstances similar to those that paved the way for the Quant Meltdown of 2007 described in Lesson 2. Among the authors’ contributions in their proposed model, neural network-based automatic factor construction (NNAFC) is a process to ensure diversity among the resulting factors.

4. Pre-Training and Prior Knowledge

To increase diversity in their model compared to standard GP, Fang et al. “pre-train” their neural network. That is, they select several indicator strategies, e.g., moving average (MA), exponential moving average (EMA), moving average convergene/divergence (MACD) for the neural net to replicate, which it does with relatively high accuracy—85% and above (Fang et al. 11). These indicator strategies compose the “domain knowledge” or “prior knowledge” of the experts. Recall the way that the Black-Litterman model incorporates expert views about returns that are not reflected in the data. By forcing the neural net to learn these strategies first in the pre-training process, they ensure that some form of these strategies persist in the to-be-developed final model, which also ensures a higher degree of diversity than traditional GP produced. The authors refer to

this process as a type of “model stealing” (Fang et al. 9), but model extraction, transfer learning, or knowledge distillation might be more apt (and common) terms for this type of technique.

Note: When the authors use the word “derivable,” they mean differentiable: A derivative of the function exists for every value in the function’s domain. For example, Gu states, “A meaningful *derivable* kernel function is proposed to replace the *un-derivable* operator rank()” (6, italics added), but they mean a *differentiable* kernel function and a *non-differentiable* operator, respectively.

5. Conclusion

In this module, we reviewed the fundamentals of factor-based investing, as well as the risks inherent to this popular strategy. We also surveyed the many approaches to factor modeling, from simple linear models to more complex models built with machine learning techniques. In this lesson, we saw how neural networks can be pre-trained before being combined with genetic programming in order to construct a superior multi-factor model. In the next module, we discuss some additional techniques and tools for improving the portfolio construction process, especially with respect to correlation and covariance matrices: how to make them more robust and less noisy.

References

- Fang, Jie, et al. "Neural Network-based Automatic Factor Construction." *Quantitative Finance*, vol. 8, no. 15, 2020, pp. 1–22.
- *Genetic Programming*. https://geneticprogramming.com/